

第32章 TIM—基本定时器

本章参考资料：《STM32F4xx 参考手册》、《STM32F4xx 规格书》、库帮助文档《stm32f4xx_dsp_stdperiph_lib_um.chm》。

学习本章时，配合《STM32F4xx 参考手册》基本定时器章节一起阅读，效果会更佳，特别是涉及到寄存器说明的部分。

特别说明，本书内容是以 STM32F4xx 系列控制器资源讲解。

32.1 TIM 简介

定时器(Timer)最基本的功能就是定时了，比如定时发送 USART 数据、定时采集 AD 数据等等。如果把定时器与 GPIO 结合起来使用的话可以实现非常丰富的功能，可以测量输入信号的脉冲宽度，可以生产输出波形。定时器生产 PWM 控制电机状态是工业控制普遍方法，这方面知识非常有必要深入了解。

STM32F42xxx 系列控制器有 2 个高级控制定时器、10 个通用定时器和 2 个基本定时器，还有 2 个看门狗定时器。看门狗定时器不在本章讨论范围，有专门讲解的章节。控制器上所有定时器都是彼此独立的，不共享任何资源。各个定时器特性参考表 32-1。

表 32-1 各个定时器特性

定时器类型	Timer	计数器分辨率	计数器类型	预分频系数	DMA 请求生成	捕获/比较通道	互补输出	最大接口时钟(MHz)	最大定时器时钟(MHz)
高级控制	TIM1 和 TIM8	16 位	递增、递减、递增/递减	1~65536(整数)	有	4	有	84 (APB2)	168
通用	TIM2, TIM5	32 位	递增、递减、递增/递减	1~65536(整数)	有	4	无	42 (APB1)	84/168
	TIM3, TIM4	16 位	递增、递减、递增/递减	1~65536(整数)	有	4	无	42 (APB1)	84/168
	TIM9	16 位	递增	1~65536(整数)	无	2	无	84 (APB2)	168
	TIM10, TIM11	16 位	递增	1~65536(整数)	无	1	无	84 (APB2)	168
	TIM12	16 位	递增	1~65536(整数)	无	2	无	42 (APB1)	84/168
	TIM13, TIM14	16 位	递增	1~65536(整数)	无	1	无	42 (APB1)	84/168
基本	TIM6 和 TIM7	16 位	递增	1~65536(整数)	有	0	无	42 (APB1)	84/168

其中最大定时器时钟可通过 RCC_DCKCFGR 寄存器配置为 84MHz 或者 168MHz。

定时器功能强大，这一点透过《STM32F4xx 中文参考手册》讲解定时器内容就有 160 多页就显而易见了。定时器篇幅长，内容多，对于新手想完全掌握确实有些难度，特别参考手册是先介绍高级控制定时器，然后介绍通用定时器，最后才介绍基本定时器。实际上，就功能上来说通用定时器包含所有基本定时器功能，而高级控制定时器包含通用定时器所有功能。所以高级控制定时器功能繁多，但也是最难理解的，本章我们先选择最简单的基本定时器进行讲解。

32.2 基本定时器

基本定时器比高级控制定时器和通用定时器功能少，结构简单，理解起来更容易，我们就开始先讲解基本定时器内容。基本定时器主要两个功能，第一就是基本定时功能，生成时基，第二就是专门用于驱动数模转换器(DAC)。关于驱动 DAC 具体应用参考 DAC 章节。

控制器有两个基本定时器 TIM6 和 TIM7，功能完全一样，但所用资源彼此都完全独立，可以同时使用。在本章内容中，以 TIMx 统称基本定时器。

基本上定时器 TIM6 和 TIM7 是一个 16 位向上递增的定时器，当我在自动重载寄存器(TIMx_ARR)添加一个计数值后并使能 TIMx，计数寄存器(TIMx_CNT)就会从 0 开始递增，当 TIMx_CNT 的数值与 TIMx_ARR 值相同时就会生成事件并把 TIMx_CNT 寄存器清 0，完成一次循环过程。如果没有停止定时器就循环执行上述过程。这些只是大概的流程，希望大家有个感性认识，下面细讲整个过程。

32.3 基本定时器功能框图

基本定时器的功能框图包含了基本定时器最核心内容，掌握了功能框图，对基本定时器就有一个整体的把握，在编程时思路就非常清晰，见图 32-1。

首先先看图 32-1 中绿色框内容，第一个是带有阴影的方框，方框内容一般是一个寄存器名称，比如图中主体部分的自动重载寄存器(TIMx_ARR)或 PSC 预分频器(TIMx_PSC)，这里要特别突出的是阴影这个标志的作用，它表示这个寄存器还自带有影子寄存器，在硬件结构上实际是有两个寄存器，源寄存器是我们可以进行读写操作，而影子寄存器我们是完全无法操作的，有内部硬件使用。影子寄存器是在程序运行时真正起到作用的，源寄存器只是给我们读写用的，只有在特定时候(特定事件发生时)才把源寄存器的值拷贝给它的影子寄存器。多个影子寄存器一起使用可以到达同步更新多个寄存器内容的目的。

接下来是一个指向右下角的图标，它表示一个事件，而一个指向右上角的图标表示中断和 DMA 输出。这个我们把它放在图中主体更好理解。图中的自动重载寄存器有影子寄存器，它左边有一个带有“U”字母的事件图标，表示在更新事件生成时就把自动重载寄存器内容拷贝到影子寄存器内，这个与上面分析是一致。寄存器右边的事件图标、中断和 DMA 输出图标表示在自动重载寄存器值与计数器寄存器值相等时生成事件、中断和 DMA 输出。

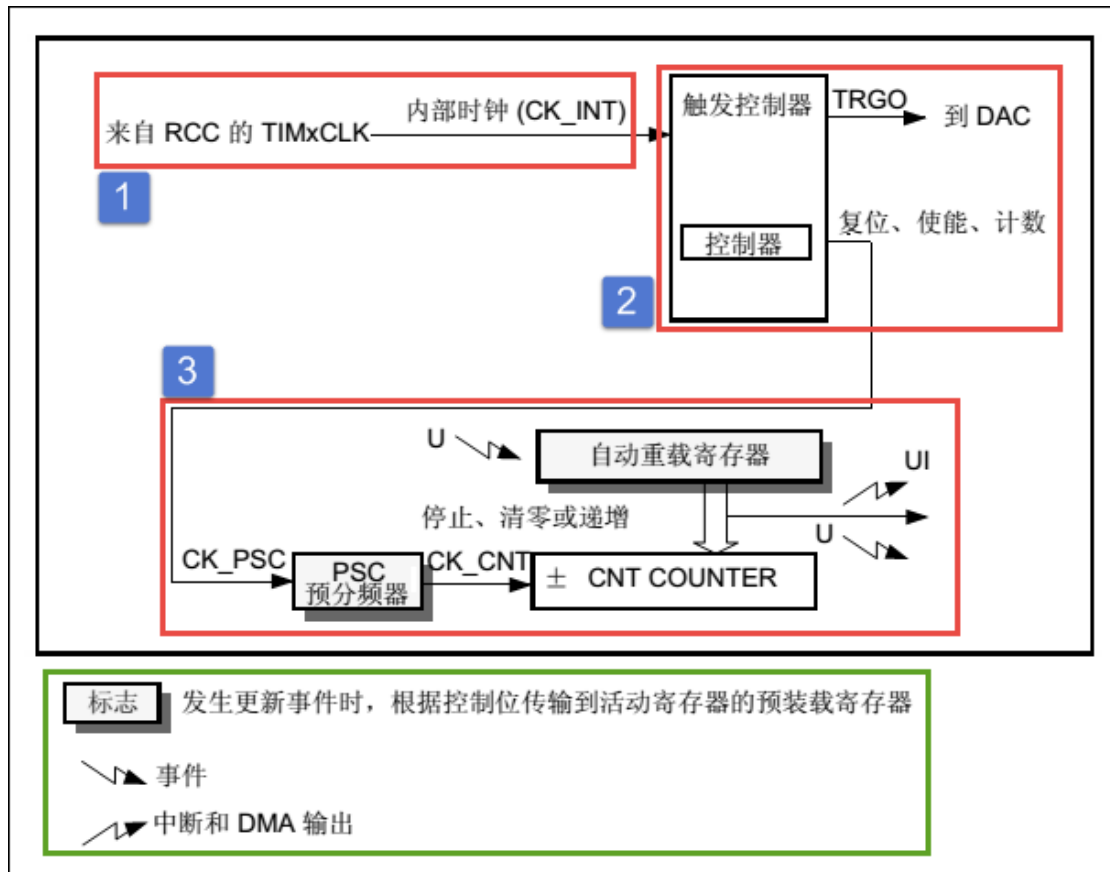


图 32-1 基本定时器功能框图

1. ①时钟源

定时器要实现计数必须有个时钟源，基本定时器时钟只能来自内部时钟，高级控制定时器和通用定时器还可以选择外部时钟源或者直接来自其他定时器等待模式。我们可以通过 RCC 专用时钟配置寄存器(RCC_DCKCFGR)的 TIMPRE 位设置所有定时器的时钟频率，我们一般设置该位为默认值 0，即 TIMxCLK 为总线时钟的两倍，使得表 32-1 中可选的最大定时器时钟为 84MHz，即基本定时器的内部时钟(CK_INT)频率为 84MHz。

基本定时器只能使用内部时钟，当 TIM6 和 TIM7 控制寄存器 1(TIMx_CR1)的 CEN 位置 1 时，启动基本定时器，并且预分频器的时钟来源就是 CK_INT。对于高级控制定时器和通用定时器的时钟源可以来找控制器外部时钟、其他定时器等等模式，较为复杂，我们在相关章节会详细介绍。

2. ②控制器

定时器控制器控制实现定时器功能，控制定时器复位、使能、计数是其基础功能，基本定时器还专门用于 DAC 转换触发。

3. ③计数器

基本定时器计数过程主要涉及到三个寄存器内容，分别是计数器寄存器(TIMx_CNT)、预分频器寄存器(TIMx_PSC)、自动重载寄存器(TIMx_ARR)，这三个寄存器都是 16 位有效数字，即可设置值为 0 至 65535。

首先我们来看图 32-1 中预分频器 PSC，它有一个输入时钟 CK_PSC 和一个输出时钟 CK_CNT。输入时钟 CK_PSC 来源于控制器部分，基本定时器只有内部时钟源所以 CK_PSC 实际等于 CK_INT，即 90MHz。在不同应用场所，经常需要不同的定时频率，通过设置预分频器 PSC 的值可以非常方便得到不同的 CK_CNT，实际计算为： f_{CK_CNT} 等于 $f_{CK_PSC}/(PSC[15:0]+1)$ 。

图 32-2 是将预分频器 PSC 的值从 1 改为 4 时计数器时钟变化过程。原来是 1 分频，CK_PSC 和 CK_CNT 频率相同。向 TIMx_PSC 寄存器写入新值时，并不会马上更新 CK_CNT 输出频率，而是等到更新事件发生时，把 TIMx_PSC 寄存器值更新到影子寄存器中，使其真正产生效果。更新为 4 分频后，在 CK_PSC 连续出现 4 个脉冲后 CK_CNT 才产生一个脉冲。

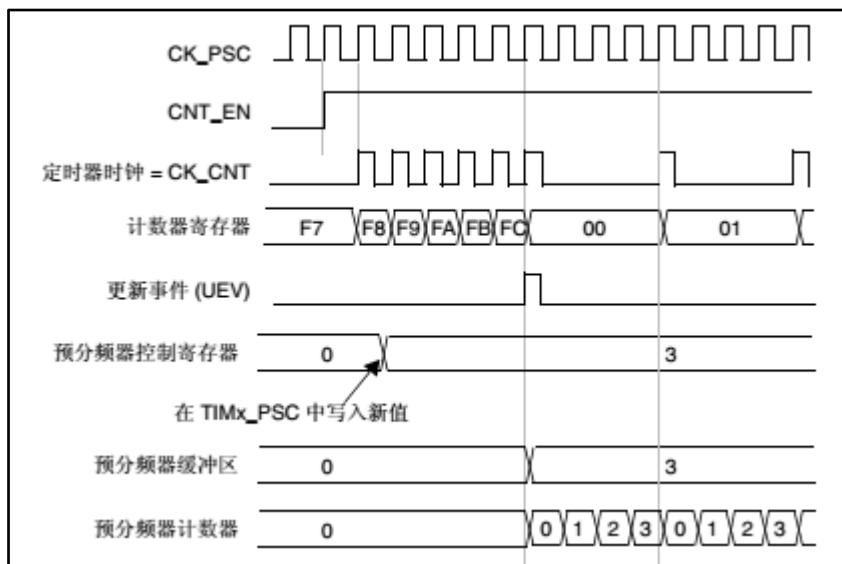


图 32-2 基本定时器时钟源分频

在定时器使能(CEN 置 1)时，计数器 COUNTER 根据 CK_CNT 频率向上计数，即每来一个 CK_CNT 脉冲，TIMx_CNT 值就加 1。当 TIMx_CNT 值与 TIMx_ARR 的设定值相等时就自动生成事件并 TIMx_CNT 自动清零，然后自动重新开始计数，如此重复以上过程。由此可见，我们只要设置 CK_PSC 和 TIMx_ARR 这两个寄存器的值就可以控制事件生成的时间，而我们一般的应用程序就是在事件生成的回调函数中运行的。在 TIMx_CNT 递增至与 TIMx_ARR 值相等，我们叫做为定时器上溢。

自动重载寄存器 TIMx_ARR 用来存放于计数器值比较的数值，如果两个数值相等就生成事件，将相关事件标志位置位，生成 DMA 和中断输出。TIMx_ARR 有影子寄存器，可以通过 TIMx_CR1 寄存器的 ARPE 位控制影子寄存器功能，如果 ARPE 位置 1，影子寄存

器有效，只有在事件更新时才把 TIMx_ARR 值赋给影子寄存器。如果 ARPE 位为 0，修改 TIMx_ARR 值马上有效。

4. 定时器周期计算

经过上面分析，我们知道定时事件生成时间主要由 TIMx_PSC 和 TIMx_ARR 两个寄存器值决定，这个也就是定时器的周期。比如我们需要一个 1s 周期的定时器，具体这两个寄存器值该如何设置内。假设，我们先设置 TIMx_ARR 寄存器值为 9999，即当 TIMx_CNT 从 0 开始计算，刚好等于 9999 时生成事件，总共计数 10000 次，那么如果此时时钟源周期为 100us 即可得到刚好 1s 的定时周期。

接下来问题就是设置 TIMx_PSC 寄存器值使得 CK_CNT 输出为 100us 周期(10000Hz)的时钟。预分频器的输入时钟 CK_PSC 为 84MHz，所以设置预分频器值为(8400-1)即可满足。

32.4 定时器初始化结构体详解

标准库函数对定时器外设建立了四个初始化结构体，基本定时器只用到其中一个即 TIM_TimeBaseInitTypeDef，该结构体成员用于设置定时器基本工作参数，并由定时器基本初始化配置函数 TIM_TimeBaseInit 调用，这些设定参数将会设置定时器相应的寄存器，达到配置定时器工作环境的目的。这一章我们只介绍 TIM_TimeBaseInitTypeDef 结构体，其他结构体将在相关章节介绍。

初始化结构体和初始化库函数配合使用是标准库精髓所在，理解了初始化结构体每个成员意义基本上就可以对该外设运用自如了。初始化结构体定义在 stm32f4xx_tim.h 文件中，初始化库函数定义在 stm32f4xx_tim.c 文件中，编程时我们可以结合这两个文件内注释使用。

代码清单 32-1 定时器基本初始化结构体

```
1 typedef struct {  
2     uint16_t TIM_Prescaler;           // 预分频器  
3     uint16_t TIM_CounterMode;         // 计数模式  
4     uint32_t TIM_Period;              // 定时器周期  
5     uint16_t TIM_ClockDivision;       // 时钟分频  
6     uint8_t TIM_RepetitionCounter;    // 重复计数器  
7 } TIM_TimeBaseInitTypeDef;
```

- (1) TIM_Prescaler: 定时器预分频器设置，时钟源经该预分频器才是定时器时钟，它设定 TIMx_PSC 寄存器的值。可设置范围为 0 至 65535，实现 1 至 65536 分频。
- (2) TIM_CounterMode: 定时器计数方式，可是在为向上计数、向下计数以及三种中心对齐模式。基本定时器只能是向上计数，即 TIMx_CNT 只能从 0 开始递增，并且无需初始化。
- (3) TIM_Period: 定时器周期，实际就是设定自动重载寄存器的值，在事件生成时更新到影子寄存器。可设置范围为 0 至 65535。
- (4) TIM_ClockDivision: 时钟分频，设置定时器时钟 CK_INT 频率与数字滤波器采样时钟频率分频比，基本定时器没有此功能，不用设置。
- (5) TIM_RepetitionCounter: 重复计数器，属于高级控制寄存器专用寄存器位，利用它可以非常容易控制输出 PWM 的个数。这里不用设置。

虽然定时器基本初始化结构体有 5 个成员，但对于基本定时器只需设置其中两个就可以，想想使用基本定时器就是简单。

32.5 基本定时器定时实验

在 DAC 转换中几乎都用到基本定时器，使用有关基本定时器触发 DAC 转换内容在 DAC 章节讲解即可，这里就利用基本定时器实现简单的定时功能。

我们使用基本定时器循环定时 0.5s 并使能定时器中断，每到 0.5s 就在定时器中断服务函数翻转 RGB 彩灯，使得最终效果 RGB 彩灯暗 0.5s，亮 0.5s，如此循环。

32.5.1 硬件设计

基本定时器没有相关 GPIO，这里我们只用定时器的定时功能，无效其他外部引脚，至于 RGB 彩灯硬件可参考 GPIO 章节。

32.5.2 软件设计

这里只讲解核心的部分代码，有些变量的设置，头文件的包含等并没有涉及到，完整的代码请参考本章配套的工程。我们创建了两个文件：bsp_basic_tim.c 和 bsp_basic_tim.h 文件用来存基本定时器驱动程序及相关宏定义，中断服务函数放在 stm32f4xx_it.h 文件中。

1. 编程要点

- (1) 初始化 RGB 彩灯 GPIO；
- (2) 开启基本定时器时钟；
- (3) 设置定时器周期和预分频器；
- (4) 启动定时器更新中断，并开启定时器；
- (5) 定时器中断服务函数实现 RGB 彩灯翻转。

2. 软件分析

宏定义

代码清单 32-2 宏定义

```
1 #define BASIC_TIM          TIM6
2 #define BASIC_TIM_CLK      RCC_APB1Periph_TIM6
3
4 #define BASIC_TIM_IRQn     TIM6_DAC_IRQn
5 #define BASIC_TIM_IRQHandler TIM6_DAC_IRQHandler
```

使用宏定义非常方便程序升级、移植。

NVIC 配置

代码清单 32-3 NVIC 配置

```
1 static void TIMx_NVIC_Configuration(void)
2 {
3     NVIC_InitTypeDef NVIC_InitStructure;
4     // 设置中断组为 0
5     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_0);
```

```
6 // 设置中断来源
7 NVIC_InitStructure.NVIC_IRQChannel = BASIC_TIM_IRQn;
8 // 设置抢占优先级
9 NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
10 // 设置子优先级
11 NVIC_InitStructure.NVIC_IRQChannelSubPriority = 3;
12 NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
13 NVIC_Init(&NVIC_InitStructure);
14 }
```

实验用到定时器更新中断，需要配置 NVIC，实验只有一个中断，对 NVIC 配置没什么具体要求。

基本定时器模式配置

代码清单 32-4 基本定时器模式配置

```
1 /*
2 *注意: TIM_TimeBaseInitTypeDef 结构体里面有 5 个成员, TIM6 和 TIM7 的寄存器里面只有
3 *TIM_Prescaler 和 TIM_Period, 所以使用 TIM6 和 TIM7 的时候只需初始化这两个成员即可,
4 * 另外三个成员是通用定时器和高级定时器才有.
5 *-----
6 * TIM_Prescaler          都有
7 * TIM_CounterMode        TIMx,x[6,7]没有, 其他都有 (基本定时器)
8 * TIM_Period             都有
9 * TIM_ClockDivision      TIMx,x[6,7]没有, 其他都有 (基本定时器)
10 * TIM_RepetitionCounter  TIMx,x[1,8]才有 (高级定时器)
11 *-----
12 */
13 static void TIM_Mode_Config(void)
14 {
15     TIM_TimeBaseInitTypeDef  TIM_TimeBaseStructure;
16
17     // 开启 TIMx_CLK
18     RCC_APB1PeriphClockCmd(BASIC_TIM_CLK, ENABLE);
19
20     /* 累计 TIM_Period 个后产生一个更新或者中断*/
21     //当定时器从 0 计数到 4999, 即为 5000 次, 为一个定时周期
22     TIM_TimeBaseStructure.TIM_Period = 5000-1;
23
24     //定时器时钟源 TIMxCLK = 2 * PCLK1
25     //      PCLK1 = HCLK / 4
26     //      => TIMxCLK=HCLK/2=SystemCoreClock/2=84MHz
27     // 设定定时器频率为=TIMxCLK/(TIM_Prescaler+1)=10000Hz
28     TIM_TimeBaseStructure.TIM_Prescaler = 8400-1;
29
30     // 初始化定时器 TIMx
31     TIM_TimeBaseInit(BASIC_TIM, &TIM_TimeBaseStructure);
32
33
34     // 清除定时器更新中断标志位
35     TIM_ClearFlag(BASIC_TIM, TIM_FLAG_Update);
36
37     // 开启定时器更新中断
38     TIM_ITConfig(BASIC_TIM, TIM_IT_Update, ENABLE);
39
40     // 使能定时器
41     TIM_Cmd(BASIC_TIM, ENABLE);
42 }
```

使用定时器之前都必须开启定时器时钟，基本定时器属于 APB1 总线外设。

接下来设置定时器周期数为 4999，即计数 5000 次生成事件。设置定时器预分频器为 (8400-1)，基本定时器使能内部时钟，频率为 84MHz，经过预分频器后得到 10KHz 的频率。然后就是调用 TIM_TimeBaseInit 函数完成定时器配置。

TIM_ClearFlag 函数用来在配置中断之前清除定时器更新中断标志位，实际是清零 TIMx_SR 寄存器的 UIF 位。

使用 TIM_ITConfig 函数配置使能定时器更新中断，即在发生上溢时产生中断。

最后使用 TIM_Cmd 函数开启定时器。

定时器中断服务函数

代码清单 32-5 定时器中断服务函数

```
1 void BASIC_TIM_IRQHandler (void)
2 {
3     if ( TIM_GetITStatus( BASIC_TIM, TIM_IT_Update) != RESET ) {
4         LED1_TOGGLE;
5         TIM_ClearITPendingBit(BASIC_TIM , TIM_IT_Update);
6     }
7 }
```

我们在 TIM_Mode_Config 函数启动了定时器更新中断，在发生中断时，中断服务函数就得到运行。在服务函数内我们先调用定时器中断标志读取函数 TIM_GetITStatus 获取当前定时器中断位状态，确定产生中断后才运行 RGB 彩灯翻转动作，并使用定时器标志位清除函数 TIM_ClearITPendingBit 清除中断标志位。

主函数

代码清单 32-6 主函数

```
1 #include "stm32f4xx.h"
2 #include "../tim/bsp_basic_tim.h"
3 #include "../led/bsp_led.h"
4
5 /**
6  * @brief 主函数
7  * @param 无
8  * @retval 无
9  */
10 int main(void)
11 {
12
13     LED_GPIO_Config();
14
15     /* 初始化基本定时器定时，0.5s 产生一次中断 */
16     TIMx_Configuration();
17
18     while (1) {
19     }
20 }
```

实验用到 RGB 彩灯，需要对其初始化配置。LED_GPIO_Config 函数是定义在 bsp_led.c 文件的完成 RGB 彩灯 GPIO 初始化配置的程序。

TIMx_Configuration 函数是定义在 bsp_basic_tim.c 文件的一个函数，它只是简单的先后调用 TIMx_NVIC_Configuration 和 TIM_Mode_Config 两个函数完成 NVIC 配置和基本定时器模式配置。

32.5.3 下载验证

保证开发板相关硬件连接正确，把编译好的程序下载到开发板。开始 RGB 彩灯是暗的，等一会 RGB 彩灯变为红色，再等一会又暗了，如此反复。如果我们使用表钟与 RGB 彩灯闪烁对比，可以发现它是每 0.5s 改变一次 RGB 彩灯状态的。